

Building and Attacking a Simplified TLS-Style Handshake

A Client/Server Key-Exchange Protocol over Sockets, and a Man-in-the-Middle Assessment of It

Student	Mhyiar Elmistere
Instructor	Ms. Nada Salaheddin Gheriyani
Course / Term	Information Security — Semester Project, Summer 2026
Project topic	TLS Handshake Walkthrough · Network security protocol · Python + sockets

1. Executive Summary

This project designs, implements, and then security-tests a simplified TLS-style handshake between a client and a server communicating over local TCP sockets. The protocol reproduces the essential stages of transport-layer security: a hello exchange with fresh random values, an ephemeral Diffie-Hellman key exchange, derivation of session keys with HKDF, and an authenticated encrypted channel built on AES-256-GCM. The goal is not to replace a production TLS library but to make the mechanics of secure channel establishment concrete and measurable.

The assessment treated the initial (unauthenticated) implementation as the system under test. A controlled man-in-the-middle experiment on the loopback interface showed that, without peer authentication, an attacker positioned between the two parties negotiates a separate key with each side and reads the entire “encrypted” conversation in plaintext while both victims observe a normal session. This is recorded as Finding-01 (High). Three further weaknesses relating to forward secrecy, nonce management, and trust bootstrapping were identified. Each finding was remediated and re-tested: adding an Ed25519-signed key exchange with client-side key pinning caused the identical attack to fail, with the client aborting the handshake on signature verification. The outcome is a working protocol, reproducible evidence, and a clear demonstration that authentication — not encryption alone — is what makes a key exchange trustworthy.

2. Introduction, Scope & Objectives

2.1 Problem statement

Two programs that simply encrypt their traffic are not secure if they cannot first agree on a key that only they know. TLS solves this with a handshake. To understand why each step exists, this project rebuilds a minimal handshake from primitives and then asks the security question directly: what happens when a step — specifically authentication — is missing? The system under test is therefore our own protocol implementation, which we are fully authorised to attack.

2.2 Objectives (mapped to the CIA triad)

- **O1 — Confidentiality:** Establish a shared session key over an untrusted channel using ephemeral Diffie-Hellman.
- **O2 — Integrity:** Protect application data with AES-256-GCM and detect any modification of ciphertext.
- **O3 — Integrity:** Bind the whole handshake with a Finished MAC so tampering with earlier messages is caught.
- **O4 — Authenticity:** Authenticate the server so the client knows who it shares the key with.
- **O5 — Verification:** Demonstrate a MITM succeeding without O4 and failing with it.

2.3 Scope and rules of engagement

- **In scope:** The project's own client, server, and protocol code only.
- **Out of scope / forbidden:** Any third-party host, production system, or network other than the local loopback interface (127.0.0.1). No traffic leaves the machine.
- **Rules of engagement:** All experiments, including the man-in-the-middle attack, run on a self-owned lab on 127.0.0.1. The attacker component is used solely to prove a weakness in code we own and then to validate its fix.

2.4 Legal & ethical statement

Testing was performed exclusively against software authored by the student and executed on the student's own machine. No unauthorised system was scanned, accessed, or attacked, and no vulnerability was disclosed to or weaponised against any third party. This satisfies the "authorization first" and "responsible disclosure" principles of the course.

3. Methodology

The work followed the professional assessment lifecycle taught in the course — threat modelling, reconnaissance, vulnerability assessment, controlled exploitation, and hardening — adapted to a protocol we implement ourselves.

3.1 Threat model

- **Adversary:** A Dolev-Yao network attacker who can read, drop, modify, and inject any message on the wire, and can position themselves between client and server (an active man-in-the-middle).
- **Assets:** The confidentiality and integrity of application data, and the authenticity of the server the client believes it is talking to.
- **Assumptions:** The endpoints themselves are trusted; side-channel, memory-corruption, and denial-of-service attacks are out of scope.

3.2 Reconnaissance & design analysis

Because the target is our own protocol, reconnaissance took the form of a design review of the message flow (ClientHello → ServerHello → ClientKeyExchange → Finished → application data) to locate the trust assumptions in each step. The review flagged the ServerHello as the point where the client accepts a key with no proof of who sent it — the hypothesis later confirmed by exploitation.

3.3 Vulnerability assessment

Each protocol property was tested in isolation with automated checks: honest parties must derive identical keys; a substituted (attacker) key must yield different keys on each side; AES-GCM must reject any modified ciphertext; and the Finished MAC must change if any handshake byte changes. All five checks pass and form the baseline for the exploitation phase.

3.4 Controlled exploitation

A man-in-the-middle proxy was placed between client and server on the loopback interface. It completes a full, independent handshake with each side, holding both session keys, and relays every record after decrypting and re-encrypting it. The plaintext it recovers is captured as evidence (Section 5). The same experiment was then repeated against the authenticated variant to verify the remediation.

3.5 CIA-triad analysis

Confidentiality and integrity of application data are provided by AES-256-GCM once a key is agreed; the Finished MAC and GCM tag defend integrity. The missing property in the baseline is authenticity: without it, confidentiality collapses because the key is agreed with the wrong party. Availability is not a focus of this protocol-level study.

4. Environment & Implementation

4.1 Lab environment

Component	Detail
Host / OS	Single laptop, macOS; all sockets bound to 127.0.0.1 (loopback only)
Language / runtime	Python 3.11
Cryptography library	pyca/cryptography 42.x (AES-256-GCM, Ed25519)
Standard library	socket, hashlib, hmac, os.urandom (DH, HKDF, framing)
Cipher suite	DHE-HKDF-SHA256-AES256GCM (single negotiated suite)
Key exchange group	RFC 3526 2048-bit MODP (Group 14); generator $g = 2$
Server identity	Long-term Ed25519 key, pinned by the client in secure mode

4.2 Architecture & message flow

```
CLIENT                                SERVER
|----- ClientHello (Nc, suites) ---->|
|<----- ServerHello (Ns, g^b [,sig])--|
|----- ClientKeyExchange (g^a) ---->|
|   K = HKDF( (g^b)^a , Nc||Ns )       |   K = HKDF( (g^a)^b , Nc||Ns )
|----- Finished (HMAC transcript) -->|
|<----- Finished (HMAC transcript) ---|
|==== AES-256-GCM application data ===|
Nc,Ns = 32-byte randoms   sig = Ed25519 over (Nc||Ns||g^b), secure mode only
```

4.3 Code architecture

- **tls_common.py** — DH group, HKDF-extract/expand, AES-GCM record layer, running transcript hash, Ed25519 sign/verify, and length-prefixed JSON framing.
- **tls_server.py** — performs the six handshake steps; a `--secure` flag signs the DH public key.
- **tls_client.py** — mirrors the steps; a `--secure` flag verifies the signature and pins the server key, aborting on mismatch.
- **mitm.py** — the man-in-the-middle proxy used for exploitation (lab only).
- **run_demo.py** — one-command driver that runs all four scenarios and produces the evidence transcript.

The full source and evidence are in the project repository (see Section 6, References).

5. Findings & Analysis

Findings are numbered and rated using CVSS 3.1 base scores. Each lists concrete evidence from the captured demo transcript and the remediation that was implemented and re-tested.

ID	Finding	Severity (CVSS 3.1)	Status
F-01	Unauthenticated key exchange enables a full man-in-the-middle: the channel key is agreed with no proof of the peer's identity.	High — 7.4 AV:N/AC:H/PR:N/UI:N/ S:U/C:H/I:H/A:N	Fixed & re-tested
F-02	No forward secrecy if long-term DH keys were reused: compromise of one private value would expose past sessions.	Medium — 5.9	Mitigated by design
F-03	AES-GCM (key, nonce) reuse would be catastrophic, voiding confidentiality and integrity.	Medium — 5.9	Mitigated by design
F-04	Trust is bootstrapped by manual key pinning; there is no certificate/PKI chain of trust.	Low — 3.7	Accepted / future work

Finding-01 — Missing peer authentication (MITM). Evidence:

```
[CLIENT] shared secret      = 89b897dd56899d8aa219050ec94c6698...(256B)
[MITM]   leg-A shared secret= 89b897dd56899d8aa219050ec94c6698...(256B) <- identical
[MITM]   !!! DECRYPTED victim -> server traffic in the clear:
          stolen plaintext = 'Mihyar'
[MITM]   !!! DECRYPTED server -> victim traffic in the clear:
          stolen plaintext = 'Hello Mihyar - this channel is AES-256-GCM encrypted.'
```

Analysis: the attacker's key with the victim matches the victim's key exactly, confirming the victim negotiated with the attacker, not the server. Both sides believed the session succeeded.

Remediation & re-test:

The server now signs (client_random || server_random || g^b) with a long-term Ed25519 key, and the client verifies that signature against a pinned public key. Re-running the identical attack:

```
[SERVER] secure mode: signed DH public key with long-term Ed25519 key
[CLIENT] !! HANDSHAKE ABORTED: Invalid server signature - possible MITM!
[MITM]   Victim closed the connection -> attack DEFEATED (server was authenticated)
```

Result: the attacker cannot forge the signature for its substituted key, so the client aborts. Findings F-02 and F-03 are mitigated by design (ephemeral per-session DH keys; a monotonic per-record sequence-number nonce for GCM). F-04 is accepted as a scoped limitation.

6. Recommendations, Limitations & References

6.1 Hardening roadmap

- **Short term:** Enable authenticated mode (--secure) so every key exchange is signed and pinned; never ship the unauthenticated baseline.
- **Medium term:** Replace raw-key pinning with X.509 certificates and a validated chain of trust; add downgrade protection across cipher suites; enforce record-count limits with rekeying.
- **Long term:** Migrate the key exchange to X25519 (ECDHE) and adopt the TLS 1.3 key schedule; consider post-quantum hybrid key exchange for long-lived confidentiality.

6.2 Limitations (honest scope)

This is a teaching implementation. It does not implement certificate/PKI validation, TLS 1.3 record padding, session resumption/0-RTT, or negotiation across multiple cipher suites, and it uses finite-field Diffie-Hellman rather than X25519. The MITM assessment covers the handshake and record layer only; it does not evaluate side channels, timing attacks, or denial of service. These are deliberate boundaries so the core key-exchange concepts remain the focus.

6.3 Conclusion

A working TLS-style handshake was built from primitives and then broken and fixed under controlled conditions. The central lesson is measurable rather than asserted: strong encryption over an unauthenticated key exchange gives a false sense of security, because an active attacker simply becomes the party you share the key with. Authenticating the key exchange — the single change from the insecure to the secure variant — is what converts a breakable channel into a trustworthy one.

6.4 References

- [1] T. Dierks and E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.2," RFC 5246, IETF, 2008.
- [2] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, IETF, 2018.
- [3] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, IETF, 2010.
- [4] T. Kivinen and M. Kojo, "MODP Diffie-Hellman groups for IKE," RFC 3526, IETF, 2003.
- [5] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, 2007.
- [6] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, IETF, 2017.
- [7] FIRST, "Common Vulnerability Scoring System v3.1: Specification Document," 2019.
- [8] Project source code and evidence repository: <https://github.com/Mhyiar98/tls-handshake-project>